# *10g.* SIMD Packages

Martin Alfaro
PhD in Economics

## INTRODUCTION

Up to this point, we've leaned on the built-in `@simd` macro to encourage vectorization in for-loops. This approach, nonetheless, exhibits several limitations.

The first one is control. The `@simd` macro acts as a suggestion, rather than a strict command: it hints to the compiler that SIMD optimizations might improve performance, but the implementation decision is ultimately up to the compiler's discretion. Second, `@simd` is designed to be conservative, prioritizing code safety over speed. In practice, this means it won't implement many transformations required to unlock SIMD's full capabilities.

To overcome these shortcomings, we'll introduce the `@turbo` macro from the `LoopVectorization` package. Unlike `@simd`, this macro can be used in both for-loops and broadcast operations.

Rather than merely suggesting vectorization, `@turbo` rewrites for-loops and broadcast expressions into a form that's explicitly structured for SIMD execution. Furthermore, it applies more aggressive optimizations and integrates with the `SLEEFPirates` package. The latter enables SIMD-accelerated implementations of common mathematical functions, including logarithms, exponentials, powers, and trigonometric operations.

This extra power comes with an important shift in responsibility: `@turbo` assumes that your code satisfies the conditions necessary for these transformations, placing the burden of safety and correctness on the user.

## CAVEATS ABOUT IMPROPER USE OF @TURBO

In contrast to `@simd`, applying `@turbo` demands deliberate care, as an incorrect application can silently produce wrong results. The risk stems from the additional assumptions that `@turbo` makes to enable more aggressive optimizations. In particular, `@turbo` operates under two key assumptions:

- **No out-of-bound access**: `@turbo` removes bounds checks entirely. Thus, all indices should be valid.
- **Iteration order invariance**: `@turbo` supposes that the outcome doesn't depend on the order in which iterations execute. Note, though, that reductions are allowed.

The second assumption becomes especially important, since it may conflict with for-loops that have dependent iterations. The example below illustrates this issue: because each iteration consumes the result produced by the previous one, applying `@turbo` breaks the required iteration-order invariance and therefore produces incorrect results.

**NO MACRO**

```julia
x = [0.1, 0.2, 0.3]

function foo!(x)
    for i in 2:length(x)
        x[i] = x[i-1] + x[i]
    end
end
```

```
julia> foo!(x)

julia> x
3-element Vector{Float64}:
 0.1
 0.3
 0.6
```

**@SIMD**

```julia
x = [0.1, 0.2, 0.3]

function foo!(x)
    @inbounds @simd for i in 2:length(x)
        x[i] = x[i-1] + x[i]
    end
end
```

```
julia> foo!(x)

julia> x
3-element Vector{Float64}:
 0.1
 0.3
 0.6
```

**@TURBO**

```julia
x = [0.1, 0.2, 0.3]

function foo!(x)
    @turbo for i in 2:length(x)
        x[i] = x[i-1] + x[i]
    end
end
```

```
julia> foo!(x)

julia> x
3-element Vector{Float64}:
 0.1
 0.3
 0.5
```

Even when a for-loop has no explicit loop-carried dependency, floating-point computations remain sensitive to reordering. In particular, because **floating-point addition isn't associative**, reordering a sequence of additions can change the final result. Note that integer arithmetic, by contrast, is unaffected by such reordering.

Considering that `@turbo` isn't suitable for all operations, we next present cases where the macro can be safely applied.

## SAFE APPLICATIONS OF @TURBO

There are two categories of operations that represent safe uses of `@turbo`: **embarrassingly parallel problems** and **reductions**.

In the first category, **each iteration is completely independent**, making execution order irrelevant for the final outcome. The example below illustrates this scenario, by performing an independent transformation on each element of a vector.

**DEFAULT**

```
x              = rand(1_000_000)
calculation(a) = a * 0.1 + a^2 * 0.2 - a^3 * 0.3 - a^4 * 0.4

function foo(x)
    output     = similar(x)

    for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.840 ms (3 allocations: 7.629 MiB)
```

**@SIMD**

```
x              = rand(1_000_000)
calculation(a) = a * 0.1 + a^2 * 0.2 - a^3 * 0.3 - a^4 * 0.4

function foo(x)
    output     = similar(x)

    @inbounds @simd for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  4.096 ms (3 allocations: 7.629 MiB)
```

**@TURBO (FOR-LOOP)**

```
x              = rand(1_000_000)
calculation(a) = a * 0.1 + a^2 * 0.2 - a^3 * 0.3 - a^4 * 0.4

function foo(x)
    output     = similar(x)

    @turbo for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  271.104 µs (3 allocations: 7.629 MiB)
```

**@TURBO (BROADCASTING)**

```
x              = rand(1_000_000)
calculation(a) = a * 0.1 + a^2 * 0.2 - a^3 * 0.3 - a^4 * 0.4

foo(x)         = @turbo calculation.(x)
```

```
julia> @btime foo($x)
  482.698 µs (3 allocations: 7.629 MiB)
```

The second safe application is **reductions**. While reductions introduce dependencies across iterations, the package only requires that iterations can be arbitrarily reordered. This is satisfied by reduction operations.

**DEFAULT**

```
x              = rand(1_000_000)
calculation(a) = a * 0.1 + a^2 * 0.2 - a^3 * 0.3 - a^4 * 0.4

function foo(x)
    output     = 0.0

    for i in eachindex(x)
        output += calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.892 ms (0 allocations: 0 bytes)
```

**@SIMD**

```
x              = rand(1_000_000)
calculation(a) = a * 0.1 + a^2 * 0.2 - a^3 * 0.3 - a^4 * 0.4

function foo(x)
    output     = 0.0

    @inbounds @simd for i in eachindex(x)
        output += calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.937 ms (0 allocations: 0 bytes)
```

**@TURBO**

```
x              = rand(1_000_000)
calculation(a) = a * 0.1 + a^2 * 0.2 - a^3 * 0.3 - a^4 * 0.4

function foo(x)
    output     = 0.0

    @turbo for i in eachindex(x)
        output += calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  179.364 µs (0 allocations: 0 bytes)
```

# ACCELERATED VERSIONS OF CERTAIN FUNCTIONS

Another advantage of `LoopVectorization` comes from its integration with the library *SLEEF* (an anocrym for "SIMD Library for Evaluating Elementary Functions"). This feature accelerates the evaluation of several mathematical functions, including the exponential, logarithmic, power, and trigonometric functions. SLEEF is exposed in `LoopVectorization` via the `SLEEFPirates` package,

Below, we illustrate the use of `@turbo` across these different functions. For a complete list of supported functions, consult the SLEEFPirates documentation. All examples are based on an element-wise transformation of `x` via the function `calculation`, which will take different definitions depending on the function illustrated.

## LOGARITHM

**DEFAULT**

```
x              = rand(1_000_000)
calculation(a) = log(a)

function foo(x)
    output     = similar(x)

    for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.542 ms (3 allocations: 7.629 MiB)
```

**@SIMD**

```
x              = rand(1_000_000)
calculation(a) = log(a)

function foo(x)
    output     = similar(x)

    @inbounds @simd for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.546 ms (3 allocations: 7.629 MiB)
```

**@TURBO (FOR-LOOP)**

```
x               = rand(1_000_000)
calculation(a) = log(a)

function foo(x)
    output      = similar(x)

    @turbo for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  1.617 ms (3 allocations: 7.629 MiB)
```

**@TURBO (BROADCASTING)**

```
x               = rand(1_000_000)
calculation(a) = log(a)

foo(x) = @turbo calculation.(x)
```

```
julia> @btime foo($x)
  1.618 ms (3 allocations: 7.629 MiB)
```

## EXPONENTIAL FUNCTION

**DEFAULT**

```
x               = rand(1_000_000)
calculation(a) = exp(a)

function foo(x)
    output      = similar(x)

    for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  2.608 ms (3 allocations: 7.629 MiB)
```

**@SIMD**

```
x              = rand(1_000_000)
calculation(a) = exp(a)

function foo(x)
    output     = similar(x)

    @inbounds @simd for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  2.639 ms (3 allocations: 7.629 MiB)
```

**@TURBO (FOR-LOOP)**

```
x              = rand(1_000_000)
calculation(a) = exp(a)

function foo(x)
    output     = similar(x)

    @turbo for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  555.012 µs (3 allocations: 7.629 MiB)
```

**@TURBO (BROADCASTING)**

```
x              = rand(1_000_000)
calculation(a) = exp(a)

foo(x) = @turbo calculation.(x)
```

```
julia> @btime foo($x)
  544.043 µs (3 allocations: 7.629 MiB)
```

## POWER FUNCTIONS

This includes any operation comprising terms $x^y$.

**DEFAULT**

```
x              = rand(1_000_000)
calculation(a) = a^4

function foo(x)
    output     = similar(x)

    for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.517 ms (3 allocations: 7.629 MiB)
```

**@SIMD**

```
x              = rand(1_000_000)
calculation(a) = a^4

function foo(x)
    output     = similar(x)

    @inbounds @simd for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.578 ms (3 allocations: 7.629 MiB)
```

**@TURBO (FOR-LOOP)**

```
x              = rand(1_000_000)
calculation(a) = a^4

function foo(x)
    output     = similar(x)

    @turbo for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  371.218 µs (3 allocations: 7.629 MiB)
```

**@TURBO (BROADCASTING)**

```
x              = rand(1_000_000)
calculation(a) = a^4

foo(x) = @turbo calculation.(x)
```

```
julia> @btime foo($x)
  302.605 µs (3 allocations: 7.629 MiB)
```

The implementation for power functions includes calls to other functions, such as the one used for computing square roots.

**DEFAULT**

```
x              = rand(1_000_000)
calculation(a) = sqrt(a)

function foo(x)
    output     = similar(x)

    for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  1.159 ms (3 allocations: 7.629 MiB)
```

**@SIMD**

```
x              = rand(1_000_000)
calculation(a) = sqrt(a)

function foo(x)
    output     = similar(x)

    @inbounds @simd for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  1.200 ms (3 allocations: 7.629 MiB)
```

**@TURBO (FOR-LOOP)**

```
x              = rand(1_000_000)
calculation(a) = sqrt(a)

function foo(x)
    output     = similar(x)

    @turbo for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  590.429 μs (3 allocations: 7.629 MiB)
```

**@TURBO (BROADCASTING)**

```
x              = rand(1_000_000)
calculation(a) = sqrt(a)

foo(x) = @turbo calculation.(x)
```

```
julia> @btime foo($x)
  578.698 μs (3 allocations: 7.629 MiB)
```

## TRIGONOMETRIC FUNCTIONS

Among others, `@turbo` can handle the functions `sin`, `cos`, and `tan`. Below, we demonstrate its use with `sin`.

**DEFAULT**

```
x              = rand(1_000_000)
calculation(a) = sin(a)

function foo(x)
    output     = similar(x)

    for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.915 ms (3 allocations: 7.629 MiB)
```

**@SIMD**

```
x              = rand(1_000_000)
calculation(a) = sin(a)

function foo(x)
    output     = similar(x)

    @inbounds @simd for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.895 ms (3 allocations: 7.629 MiB)
```

**@TURBO (FOR-LOOP)**

```
x              = rand(1_000_000)
calculation(a) = sin(a)

function foo(x)
    output     = similar(x)

    @turbo for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  1.341 ms (3 allocations: 7.629 MiB)
```

**@TURBO (BROADCASTING)**

```
x              = rand(1_000_000)
calculation(a) = sin(a)

foo(x)         = @turbo calculation.(x)
```

```
julia> @btime foo($x)
  1.315 ms (3 allocations: 7.629 MiB)
```